

Theming

The 42 built-in themes, how the picker persists a choice, and how to override the tokens.

Shravan Goswami / <https://shravangoswami.com>

What ships

Almanac has 42 built-in themes, every one of them available in both light and dark:

- The **default family**, almanac-light and almanac-dark.
- **Twelve color families**: slate, graphite, blue, indigo, violet, sky, teal, emerald, amber, rose, crimson, fuchsia.
- **Eight signature palettes**: sepia, solarized, gruvbox, dracula, nord, onedark, tokyonight, catppuccin.

Ids are <family>-<mode>, so dracula-dark, teal-light, and almanac-dark are all valid. Every family exists in both modes, which the test suite enforces, so the light/dark toggle never has to drop a visitor into a different palette.

Configuring it

```
theme: {  
  default: "almanac-light",  
  include: "all",  
}
```

default is what a visitor sees before they pick anything. include accepts "all" or an array; array entries can be full ids or bare family names, so ["dracula", "nord-dark"] gives you both Dracula modes plus dark Nord. The almanac family is always kept regardless, so there is always something to fall back to.

The token system

A theme is a set of CSS custom properties. The default family's tokens live in the framework's stylesheet under :root and :root[data-mode="dark"]; every other theme is derived from a four-color seed (background, surface, text, accent) and emitted as an override block:

```
html[data-mode="dark"][data-theme="dracula-dark"] {  
  --bg: #282a36;  
  --surface: #343746;  
  --surface-muted: #484a57;  
  --text: #f8f8f2;  
  --muted: #a5a6a7;  
  --line: #575a65;  
  --accent: #bd93f9;  
  --accent-strong: #e1cef9;  
  /* ... */  
}
```

Deriving the rest of the token set from four colors is why adding a theme is a handful of colors instead of a dozen hand-tuned properties. Every component reads these variables, so overriding --accent in one place retints the whole site.

Only the themes you included are emitted, and they are inlined in a single <style> in the head, so switching themes never fetches anything.

The picker

The header has two controls: a quick toggle and a caret that opens the picker.

- The **quick toggle** flips to the same family's opposite mode, so nord-light becomes nord-dark rather than jumping to the default palette.
- The **picker** has a System / Light / Dark segment and a grid of preview tiles, one per included theme in the current mode.

A choice is written to `localStorage` under the key `theme`, holding either a theme id or the literal `system`. Under `system`, the site follows `prefers-color-scheme` and reacts live when the OS setting changes.

To avoid a flash of the wrong palette, an inline script in the head runs before first paint: it reads the stored value, resolves `system` against the media query, and sets `data-mode` and `data-theme` on `<html>` before the rest of the page renders.

The theme-color meta tag

You don't configure `<meta name="theme-color">`. Almanac derives it from your default theme: the light value comes from that family's accent, the dark value from its background. Change `theme.default` and the color a mobile browser tints its own chrome with follows automatically.

Overriding the tokens

The tokens are plain custom properties, so your own stylesheet can redefine any of them. Match the framework's specificity when you target a specific theme:

```
html[data-mode="light"][data-theme="almanac-light"] {
  --accent: #7c3aed;
}
```

``theme.customCss`` is accepted by the config schema, but the framework does not layer those files in yet. Until it does, add your stylesheet the normal Astro way, by importing it from a page or component you own.